

Lecture 1: Linear Programming OPT

*Instructor: Sourav Chakraborty**Scribe: Atim Kumar Sasmal*

1. Linear Programming and Dual

Primal Problem:

$$\begin{array}{ll}\max & 3x + 8y + 4z \\ \text{s.t.} & x + y \leq 1 \\ & x + 3y + 2z \leq 2 \\ & 5y + z \leq 2.5 \\ & z \leq 4 \\ & x, y, z \geq 0\end{array}$$

Let dual variables be $\alpha_1, \alpha_2, \alpha_3, \alpha_4$.

Dual Problem:

$$\begin{array}{ll}\min & \alpha_1 + 2\alpha_2 + 2.5\alpha_3 + 4\alpha_4 \\ \text{s.t.} & \alpha_1 + \alpha_2 \geq 3 \\ & \alpha_1 + 3\alpha_2 + 5\alpha_3 \geq 8 \\ & 2\alpha_2 + \alpha_3 + \alpha_4 \geq 4 \\ & \alpha_i \geq 0\end{array}$$

2. Network Flow

Given a graph $G(V, E)$ with capacity $c : E \rightarrow \mathbb{R}$.

Max Flow:

$$\max \sum_u f(s, u)$$

Constraints:

$$f(u, v) \leq c(u, v)$$

$$\sum_u f(u, v) = \sum_u f(v, u), \quad \forall v \in V \setminus \{s, t\}$$

Theorem:

$$\text{Max Flow} = \text{Min Cut}$$

If capacities are integers $\Rightarrow$ integer flow exists.

3. LP Formulation of Min-Cut

$$\min \sum_{(u,v) \in E} c(u, v) y(u, v)$$

Constraints:

$$0 \leq y(u, v) \leq 1$$

$$y(u, v) \geq x_u - x_v$$

$$x_s = 1, \quad x_t = 0$$

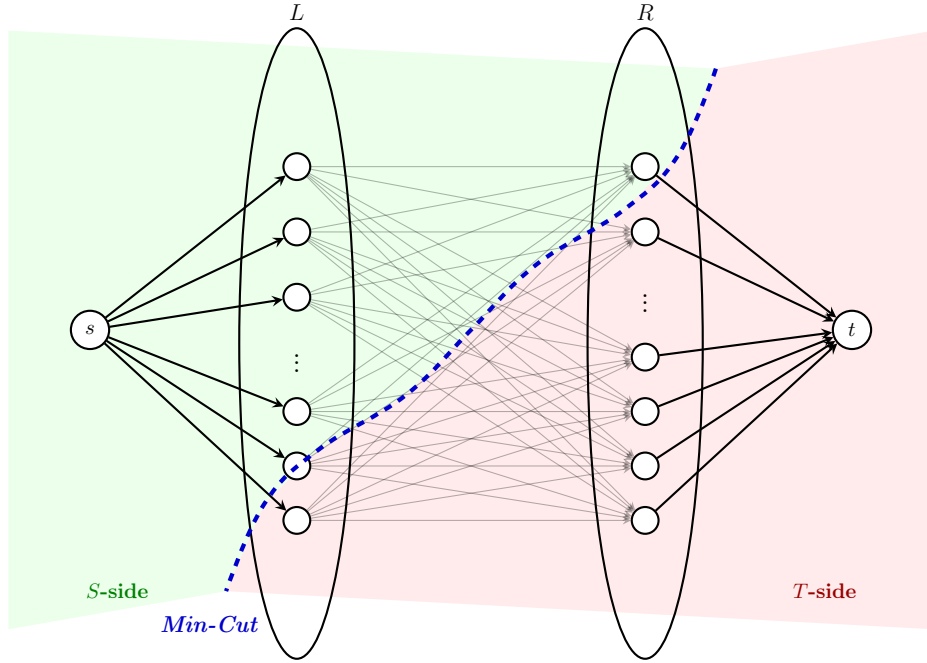
4. Bipartite Graph: L, R

Min-Cut

- Matching exists
- Perfect matching exists

Try to convert this to network flow:

$$\max \text{ flow} = |L| - |A| + |\Gamma(A)| \leq \sum_{|A|} \Rightarrow N(S) \leq |S|$$



Obs:

$$\nexists \text{ perfect matching} \iff \max \text{ flow}_K < |L|$$

Obs-1: There should be no cut between L to R , where K edges. As we are in integer flow, after integer flow will be known.

$$\min \text{ cut} = K$$

Suppose: There are b from L and some from R , i.e. d .

§ 3.1: Have $b > d$, then

$$d + |L| - b = K \geq |L|$$

$$\Leftarrow b \leq d$$

@ Which path gives you max perfect matching.

Result:

Maximum Matching = Maximum Flow

Perfect Matching Condition (Hall's Theorem):

$$\forall S \subseteq L, \quad |N(S)| \geq |S|$$

If no perfect matching:

$$\text{max flow} = k < |L|$$

Topics Covered:

1. Sorting & Searching (Analysis)
2. Greedy Algorithms
 - easy to design, hard to prove
3. Recursive Algorithms
 - Divide & Conquer
 - Dynamic Programming
4. Shortest Path
5. Modelling / Reduction
 - Reduce to known problems
 - Reduce hard problems using Heuristics Alog.
e.g. SAT, ILP
6. Complexity Theory
 - NP-Hard Problems

7. Network Flow / Matching